ETHEREUM CRYPTOGRAPHY

STUDENT: FRANCESCA PAPADIA | COURSE: CYBERSECURITY | ID: 2331411 | DATE: 13/10/2026

HMWK 1- FROM PRIVATE KEY TO PUBLIC KEY (TOY CURVE)

1. Goal

Show on a very small elliptic curve, how a private key k generates a public key Q=kG through scalar multiplication. In Ethereum the real curve is secp256k1 ($y^2 = x^3 + 7$ over a 256-bit field); here the same equation is used over a small field so that every step can be followed by hand.

2. Setup

- Curve: **$y^2 = x^3 + 7$ (mod 17)**
- Base point: **G=(1,5) (check: $1^3 + 7 = 8$ and $5^2 = 25 \equiv 8$ mod 17)**
- Private key: **k=6**
- Point addition: ($P \neq Q$): $\lambda = (y_2 - y_1)/(x_2 - x_1)$; doubling ($P = Q$): $\lambda = 3x_1^2/(2y_1)$; then $x_3 = \lambda^2 - x_1 - x_2$ and $y_3 = \lambda(x_1 - x_3) - y_1$, all mod 17. Divisions are multiplications by the modular inverse.
- The curve has 17 points. **The order of G is 9**: 9G=0.

3. Results

i	iG
1	(1,5)
2	(2,10)
3	(5,9)
4	(12,1)
5	(12,16)
6	(5,8)

TOY PUBLIC KEY: Q=6G= (5,8). Check: $5^3 + 7 = 132 \equiv 13$ and $8^2 = 64 \equiv 13$ (mod 17).

For a large k, double-and-add is used: scan k in binary, double at every bit and add G when the bit is 1. For k=6= 110_2: bit 1 → G; double → 2G; bit 1 → 3G; double → 6G. it costs O (log k) operations instead of k. the attached program (file crypto-homework.html) computes both methods, compares them and shows a step-by-step animation of the points on the curve.

4. Statistical exploration

Expretiment with k=1…60 and G=(1,5):

- **Periodicity:** iG repeats every 9 steps, so there are only 8 non-trivial public keys. The key space is tiny: recovering k from Q takes a miment by trial.
- **Distribution of x and y:** x takes only 4 values (1,2,5,12), each about 13-14 times; y takes 8 values (1,5,7,8,9,10,12,16) each about 6-7 times. Inside the subgroup the frequency is uniform, but the points do not cover the field (0…16) uniformly.
- **Bit distribution:** (fraction of 1s in the 5 bits of x): 0.50, 0.24, 0.52, 0.26, 0.00; for y:0.50, 0.24, 0.35, 0.50, 0.13. far from the ideal 0.50 for several bits, so not uniform.
- **Correlation between k and x:** 0.053, about zero. No linear correlation, but the relation is deterministic and periodic (not random).
- **Avalanche effect:** flipping one bit of k changes on average 4.36 of the 10 bits of (x,y) (ideal for a hash ≈ 5). The value is close to a hash's only by chance, given the tiny number of points, and has no cryptographic meaning.
- **Homomorphism:** (a+b)G= aG + bG. Example: 3G + 5G= (5,9) + (12,16)= (1,12)= 8G. A hash has no such structure.

5. Discussion: One-Way Function vs Hash Function

Scalar multiplication is a **trapdoor one-way function** based on the Elliptic Curve Discrete Logarithm Problem (ECDLP): k → Q= kG is easy, Q→ k is hard.

Properties that are **meaningful for a public-key one-way function**: hardness of inversion (ECDLP), determinism, efficient forward computation (double-and-add), and no algebraic shortcut to recover k from Q.

Properties **typical of cryptographic hash functions**: avalanche effect, uniform output bits, collision and second-preimage resistance, arbitrary-length input with fixed-length output, and no algebraic structure.

KG is **not** a hash: it has algebraic structure (homomorphism) and no avalanche, and that structure is exactly what enables digital signatures and verification with the public key. In the toy example inversion is trivial only because the curve is tiny.